U25AI055

Lab-8

1. **Reverse a queue**

```
//1. Reverse queue
#include<iostream>
#include<stack>
using namespace std;
struct Node
{
  int data;
  Node* next;
};
void enqueue(Node **front,Node **rear,int item);
int dequeue(Node **front,Node **rear);
void reverse(Node **front,Node **rear);
int main()
{
  Node *front=NULL,*rear=NULL;
  int choice,item;
  Node *p;
  do
  {
    cout<<"1. Enqueue\n";
    cout<<"2. Dequeue\n";
    cout<<"3. Display elements\n";
    cout<<"4. Reverse\n";
    cout<<"Enter choice: ";
    cin>>choice;
    switch(choice)
    {
      case 1:
        cout<<"Enter element: ";
        cin>>item;
        enqueue(&front,&rear,item);
        break;
      case 2:
        item=dequeue(&front,&rear);
        if(item!=-1) cout<<"Dequeued element="<<item<<endl;
        else cout<<"Queue full\n";
        break;
```

```
        case 3:
          p=front;
          while(p!=NULL)
          {
            cout<<p->data<<"\t";
            p=p->next;
          }
          cout<<endl;
          break;
        case 4:
          reverse(&front,&rear);
          break;
        default:
          cout<<"Invalid choice\n";
      }
    } while(choice>=1 && choice<=4);
    return 0;
}
void enqueue(Node **front,Node **rear,int item)
{
    Node *new_node;
    new_node=new Node;
    if(new_node==NULL) return;
    new_node->data=item;
    new_node->next=NULL;
    if(*front==NULL && *rear==NULL)
    {
      *front=new_node;
      *rear=new_node;
    }
    else
    {
      (*rear)->next=new_node;
      *rear=new_node;
    }
}
int dequeue(Node **front,Node **rear)
{
    if(*front==NULL) return -1;
    int item=(*front)->data;
    Node *temp=*front;
```

```
    if(*front==*rear)
    {
        *front=NULL;
        *rear=NULL;
    }
    else *front=(*front)->next;
    delete temp;
    return item;
}
void reverse(Node **front,Node **rear)
{
    stack<int> s1;
    int item;
    while(true)
    {
        item=dequeue(front,rear);
        if(item==-1) break;
        s1.push(item);
    }
    while(!s1.empty())
    {
        item=s1.top();
        s1.pop();
        enqueue(front,rear,item);
    }
}
```

Output:

```
1. Enqueue
2. Dequeue
3. Display elements
4. Reverse
Enter choice: 1
Enter element: 1
1. Enqueue
2. Dequeue
3. Display elements
4. Reverse
Enter choice: 1
Enter element: 2
1. Enqueue
2. Dequeue
3. Display elements
4. Reverse
Enter choice: 1
Enter element: 3
```

```
1. Enqueue
2. Dequeue
3. Display elements
4. Reverse
Enter choice: 1
Enter element: 4
1. Enqueue
2. Dequeue
3. Display elements
4. Reverse
Enter choice: 3
1       2       3       4
1. Enqueue
2. Dequeue
3. Display elements
4. Reverse
Enter choice: 4
1. Enqueue
2. Dequeue
3. Display elements
4. Reverse
Enter choice: 3
4       3       2       1
```

2. **Count number of elements in queue**

```
//2. Count number of elements in queue
#include<iostream>
#define N 10
using namespace std;
void enqueue(int *queue,int *front,int *rear,int item);
int dequeue(int *queue,int *front,int *rear);
int main()
{
  int *queue=new int[N];
  int front=-1,rear=-1;
  int choice,item;
  do
  {
    cout<<"1. Enqueue\n";
    cout<<"2. Dequeue\n";
    cout<<"3. Display elements\n";
    cout<<"4. Count number of elements\n";
    cout<<"Enter choice: ";
    cin>>choice;
    switch(choice)
    {
      case 1:
        cout<<"Enter element: ";
        cin>>item;
        enqueue(queue,&front,&rear,item);
```

```
            break;
        case 2:
            item=dequeue(queue,&front,&rear);
            if(item!=-1) cout<<"Dequeued element="<<item<<endl;
            else cout<<"Queue full\n";
            break;
        case 3:
            for(int i=front;i<=rear && i!=-1;i++) cout<<queue[i]<<"\t";
            cout<<endl;
            break;
        case 4:
            cout<<"Number of elements="<<rear-front+1<<endl;
            break;
        default:
            cout<<"Invalid choice\n";
      }
   } while(choice>=1 && choice<=4);
   cout<<"Number of elements="<<rear-front+1<<endl;
   return 0;
}
void enqueue(int *queue,int *front,int *rear,int item)
{
   if(*rear==N-1) return;
   if(*front==-1 && *rear==-1) (*front)++;
   queue[++*rear]=item;
}
int dequeue(int *queue,int *front,int *rear)
{
   if(*front==-1) return -1;
   if(*front==*rear)
   {
      *front=-1;
      *rear=-1;
   }
   return queue[(*front)++];
}
```

Output:

```
1. Enqueue
2. Dequeue
3. Display elements
4. Count number of elements
Enter choice: 1
Enter element: 12
1. Enqueue
2. Dequeue
3. Display elements
4. Count number of elements
Enter choice: 1
Enter element: 23
1. Enqueue
2. Dequeue
3. Display elements
4. Count number of elements
Enter choice: 1
Enter element: 34
```

```
1. Enqueue
2. Dequeue
3. Display elements
4. Count number of elements
Enter choice: 1
Enter element: 45
1. Enqueue
2. Dequeue
3. Display elements
4. Count number of elements
Enter choice: 4
Number of elements=4
```

3. **Find maximum and minimum element in queue**

```
//3. Find maximum and minimum element in queue
#include<iostream>
#include<climits>
using namespace std;
struct Node
{
  int data;
  Node* next;
};
struct Queue
{
  Node *front;
  Node *rear;
};
void enqueue(Queue *queue,int item);
int dequeue(Queue *queue);
int maximum(Queue *queue);
int minimum(Queue *queue);
int main()
{
  Queue *queue=new Queue;
  queue->front=NULL;
  queue->rear=NULL;
  int choice,item;
  Node *p;
  do
  {
    cout<<"1. Enqueue\n";
    cout<<"2. Dequeue\n";
```

```
        cout<<"3. Display elements\n";
        cout<<"4. Maximum element\n";
        cout<<"5. Minimum element\n";
        cout<<"Enter choice: ";
        cin>>choice;
        switch(choice)
        {
          case 1:
            cout<<"Enter element: ";
            cin>>item;
            enqueue(queue,item);
            break;
          case 2:
            item=dequeue(queue);
            if(item!=-1) cout<<"Dequeued element="<<item<<endl;
            else cout<<"Queue full\n";
            break;
          case 3:
            p=queue->front;
            while(p!=NULL)
            {
              cout<<p->data<<"\t";
              p=p->next;
            }
            cout<<endl;
            break;
          case 4:
            cout<<"Maximum element="<<maximum(queue)<<endl;
            break;
          case 5:
            cout<<"Minimum element="<<minimum(queue)<<endl;
            break;
          default:
            cout<<"Invalid choice\n";
        }
    } while(choice>=1 && choice<=5);
    return 0;
}
void enqueue(Queue *queue,int item)
{
    Node *new_node;
```

```
    new_node=new Node;
    if(new_node==NULL) return;
    new_node->data=item;
    new_node->next=NULL;
    if(queue->front==NULL && queue->rear==NULL)
    {
        queue->front=new_node;
        queue->rear=new_node;
    }
    else
    {
        (queue->rear)->next=new_node;
        queue->rear=new_node;
    }
}
int dequeue(Queue *queue)
{
    if(queue->front==NULL) return -1;
    int item=(queue->front)->data;
    Node *temp=queue->front;
    if(queue->front==queue->rear)
    {
        queue->front=NULL;
        queue->rear=NULL;
    }
    else queue->front=(queue->front)->next;
    delete temp;
    return item;
}
int maximum(Queue *queue)
{
    Node *p=queue->front;
    int max=INT_MIN;
    while(p!=NULL)
    {
        if(p->data>max) max=p->data;
        p=p->next;
    }
    return max;
}
int minimum(Queue *queue)
```

```
{
   Node *p=queue->front;
   int min=INT_MAX;
   while(p!=NULL)
   {
      if(p->data<min) min=p->data;
      p=p->next;
   }
   return min;
}
```

Output:

```
1. Enqueue
2. Dequeue
3. Display elements
4. Maximum element
5. Minimum element
Enter choice: 1
Enter element: 1
1. Enqueue
2. Dequeue
3. Display elements
4. Maximum element
5. Minimum element
Enter choice: 1
Enter element: 2
1. Enqueue
2. Dequeue
3. Display elements
4. Maximum element
5. Minimum element
Enter choice: 1
Enter element: 3
```

```
1. Enqueue
2. Dequeue
3. Display elements
4. Maximum element
5. Minimum element
Enter choice: 1
Enter element: 4
1. Enqueue
2. Dequeue
3. Display elements
4. Maximum element
5. Minimum element
Enter choice: 4
Maximum element=4
1. Enqueue
2. Dequeue
3. Display elements
4. Maximum element
5. Minimum element
Enter choice: 5
Minimum element=1
```

4. **Check palindrome using queue**

```
//4. Check palindrome using deque
#include<iostream>
using namespace std;
struct Node
{
   int data;
   Node *next;
};
void insert_front(Node **front,Node **rear,int item);
void insert_rear(Node **front,Node **rear,int item);
int delete_front(Node **front,Node **rear);
int delete_rear(Node **front,Node **rear);
bool is_palindrome(Node **front,Node **rear,char *str);
```

```
int main()
{
   Node *front=NULL,*rear=NULL;
   char str[30];
   cout<<"Enter string: ";
   cin>>str;
   if(is_palindrome(&front,&rear,str)) cout<<"Palindrome\n";
   else cout<<"Not palindrome\n";
   return 0;
}
void insert_front(Node **front,Node **rear,int item)
{
   Node *new_node=new Node;
   if(new_node==NULL) return;
   new_node->data=item;
   new_node->next=*front;
   *front=new_node;
}
void insert_rear(Node **front,Node **rear, int item)
{
   Node *new_node=new Node;
   if(new_node==NULL) return;
   new_node->data=item;
   new_node->next=NULL;
   if(*front==NULL && *rear==NULL)
   {
      *front=new_node;
      *rear=new_node;
   }
   else
   {
      (*rear)->next=new_node;
      *rear=new_node;
   }
}
int delete_front(Node **front,Node **rear)
{
   if(*front==NULL) return -1;
   int item=(*front)->data;
   if(*front==*rear)
   {
```

```
        delete *front;
        *front=NULL;
        *rear=NULL;
    }
    else
    {
        Node *temp=*front;
        *front=(*front)->next;
        delete temp;
    }
    return item;
}
int delete_rear(Node **front,Node **rear)
{
    if(*front==NULL) return -1;
    int item=(*rear)->data;
    if(*front==*rear)
    {
        *front=NULL;
        delete *rear;
        *rear=NULL;
    }
    else
    {
        Node *p=*front,*q=*front;
        while(p->next!=NULL)
        {
            q=p;
            p=p->next;
        }
        q->next=NULL;
        *rear=q;
        delete p;
    }
    return item;
}
bool is_palindrome(Node **front,Node **rear,char *str)
{
    int i=0;
    while(str[i]!='\0')
    {
```

```
        insert_rear(front,rear,str[i]);
        i++;
    }
    while(*front!=NULL && *rear!=NULL)
    {
        if(*front==*rear || (*front)->next==*rear) break;
        int f=delete_front(front,rear),r=delete_rear(front,rear);
        if(f!=r) return false;
    }
    return true;
}
```

Output:

```
Enter string: madam
Palindrome
```